G4DA COMPETITION · PAPER TRACK SUBMISSION · EXPERIMENTAL METHODOLOGY & AGENT TELEMETRY

qb-cubie in a Test-Guided Coding Agent: Methodology, Development, and Exploratory Pilot Findings

A Cryptographically Frozen A/B Comparison Between Example-Based Testing (qb-cubie 0.0.3) and pytest Across Single-Agent (v1–v5) and Multi-Agent (v6–v9) Architectures

Author & Project Lead: Dominick Wirzba · **Paper Synthesis Origin:** Gemini-App (Gemini-4-Argon) · **Post-Hoc Telemetry & Audit Analyses:** Antigravity (Gemini-3.8-Flash) & Codex (GPT-6.1-Sol)

Data Cutoff: November 12, 2026 (Single-Agent Report v1–v5 cutoff: Nov 11, 2026, archive **20261111-200641**; Multi-Agent Analysis v6–v9 cutoff: Nov 12, 2026, archive **20261112-035522**) · **Open-Source Release:** Published on PyPI (**qb-cubie**) and GitHub (MIT License).

ABSTRACT — This study investigates whether access to the declarative quality assurance framework **qb-cubie**—including its *Example-Based Testing* (EBT) engine and structured guidance—helps a test-guided coding agent resolve real-world Python modification tasks from the **SWE-bench** ecosystem under a fixed working budget compared to standard **pytest**. Both conditions operate with the same local model (**gemma4:12b**, 11.9B, GGUF Q4_K_M via Ollama 0.40.2), identical repository snapshots, and identical tools, rules, and resource limits within each pilot. An independent evaluation harness assesses exclusively the filtered production patch inside a fresh, isolated, Cubie-free Docker container against pre-frozen private FAIL_TO_PASS and PASS_TO_PASS test suites; visible tests modified by the agent cannot alter this benchmark.

Across the primary single-agent development series (generations v1 through v5; seven completed pilots comprising 42 agent runs on three SWE-bench tasks with Seed 42, backed by 5,563 verified SHA-256 hash references), privately confirmed resolutions were achieved in three runs (7.1%): twice with **pytest** (v2 and v4) and once in the **qb-cubie** condition (revised v3). Crucially, across all 21 Cubie runs in generations v1–v5, actual EBT contracts were executed in only 2 runs (9.5%), while in 19 of 21 runs (90.5%) the testing framework was never invoked at runtime due to upstream inference, editing, and pre-training prior barriers. In the v5 pilot with reasoning enabled (think=true), both conditions resolved 0 of 3 tasks: 11 of 67 model calls terminated at the output cap (num_predict=8192) emitting exclusively thinking tokens without an executable JSON action—consuming 87.3% (90,112 of 103,235) of all recorded output tokens and 79.6% of total agent runtime. Independent post-hoc root-cause analyses (Antigravity and Codex) establish that this 0:0 outcome reflects an operational **floor effect** rather than methodological equivalence. Finally, an exploratory follow-up series evaluating four serial multi-agent architectures (generations v6–v9: Analysis, QA, and Dev across 17 executed runs, 1,441 model calls, and 8.55M input tokens) demonstrates a decoupling between infrastructure maturity and workflow transition: although infrastructural errors (cursor traps, tool errors, context overflows) were reduced to 0 in v9, removing v8's forced controller handoff while overloading the orientation *finish* tool with review parameters kept all 6 v9 runs in the Analysis role (82.6% duplicate actions). Because v9 injects the **qb-cubie** guide only upon activating QA, the model never received the testing framework instructions in v9, underscoring that controller-interface coupling and upstream workflow bottlenecks must be resolved before downstream test methodologies can be evaluated.

1. Research Question, Object of Study, and Contract Semantics

The primary research question formulated in the study protocol (**Protocol, 2026**; **Single-Agent Report, 2026**) asks:

*Does access to **qb-cubie** and its specified guidance improve the independently verified resolution rate of a test-guided coding agent powered by Ollama and **gemma4:12b** under a fixed working budget compared to the exact same agent using standard **pytest**?*

What is measured is the **combined offering of qb-cubie including its documentation guide** versus **pytest** including its corresponding guide (**Study Guides, 2026**). The study does not isolate the pure library effect from the instructional prompt effect, nor does it isolate individual EBT features. When an agent fails to invoke **qb-cubie** in the treatment arm—as observed in 19 of 21 Cubie runs (90.5%) in v1–v5 and all runs in v6–v9—that non-utilization remains an empirical finding regarding the evaluated agent-and-harness package, but it precludes direct causal inferences about the efficacy of executed EBT contracts (**Single-Agent Report, 2026**; **Submission Audit, 2026**).

1.1 Declarative Contract Semantics of qb-cubie (Version 0.0.3)

The open-source package **qb-cubie** (evaluated at version 0.0.3, commit 5e50d22f80db90c4e8ae6e3aeedab6acdb981230) integrates declarative acceptance testing (*Example-Based Testing*, EBT), natural-language review delegation (*Review-Based Testing*, RBT), execution arenas (TempFS, PyVenv), unified static analysis orchestrators (AstLinter, CstLinter), and an experimental benchmark suite (qb_cubie.benchmarks / cubie-bench) with native SWE-bench integration under src/qb_cubie/benchmarks/swebench/ (**Contract Semantics, 2026**; **Benchmark Guide, 2026**; **Linter Guide, 2026**; **RBT Usage Pattern, 2026**).

In EBT, a contract specifies a finite set of concrete executions of a callable target using mutable WorldState dataclasses with fields args, kwargs, arena, fs, returns, expect, raises, and exception (**Contract Semantics, 2026**). Let P denote the declared input states in pre, Q^+ the permitted target patterns (post.expect=True), and Q^- the forbidden target patterns (post.expect=False). Where match(p, q) indicates that the outcome observed when executing p satisfies all active comparisons in q (return values, expected exceptions via ExceptionExpectation, and recursive filesystem diffs in fs), the finite evaluation rule is defined as:

```
step_ok(p) = (Q+ is empty OR at least one q in Q+ matches)
             AND (no q in Q− matches)
run_ok     = all step_ok(p) hold
evaluation = run_ok if EBT.expect=True else NOT run_ok
```

While the arrangement “pre-state – operation – post-state” serves as a didactic analogy to a Hoare triple {P} S {Q}, WorldState is a concrete data record or pattern rather than a universally proven state predicate (**Quality Assurance Guide, 2026**). Passing an EBT contract establishes *observed conformance* over finite explicit inputs rather than a formal correctness proof or coverage metric. Because every input in pre is evaluated against the entire shared target set post, correlated input/output pairs such as f(1)=2 and f(2)=4 must be executed as separate EBT contracts (e.g., via pytest.mark.parametrize) so that swapped outputs are not falsely accepted (**EBT Guide, 2026**; **EBT Usage Pattern, 2026**). Furthermore, an omitted returns field or the sentinel qb_cubie.UNSPECIFIED disables return-value checking, whereas WorldState(returns=None) explicitly requires the return value None (**Contract Semantics, 2026**). To take effect in the study, an EBT contract must be executed via .run() and its result explicitly verified (via assert result.evaluation is True when reporter=None) (**Single-Agent Report, 2026**).

1.2 Independent Technical Audit and Circularity Prevention

To uphold strict scientific transparency, the core framework was subjected to an internal critical code audit in the repository root (**Technical Audit, 2026**). While the repository test suite passes 2,569 tests (2569 passed, 18 skipped, 6 warnings in 550.50s on Linux, Python 3.13.12), the audit documented three technical defects in the core package: (B1, P1) PyVenv stores IPC files (ipc_in.pkl, ipc_out.pkl, _cubie_phantom.py) inside the workspace, which can overwrite identically named test fixtures (**Arena Management, 2026**); (B2, P1) PytestReporter relies on plain assert statements that are stripped under optimized execution (python -O); and (B3, P2) CstLinter.check() returns an empty list [] when passed a non-existent file path (**Technical Audit, 2026**). None of these three core findings affected the agent study runs.

Crucially, an early synthetic audit also highlighted the epistemological risk of circularity when validating a testing framework exclusively with its own contracts (**Technical Audit, 2026**). While the earliest test suite of **qb-cubie** originated via freehand bootstrapping combining informal RBT and EBT principles, all subsequent verification and regression suites—including the 698+ verification nodes developed for the study controllers—were deliberately written using standard, independent **pytest** assertions. Consequently, the existence of the repository’s internal test suite is not claimed as empirical evidence that AI agents write better tests with **qb-cubie**.

2. Data Foundation, SWE-bench Provenance, and Scope of Investigation

The benchmark tasks follow the established **SWE-bench** schema and originate from 129 real-world GitHub pull requests and issues across four Python repositories: 67 from FastAPI (fastapi/fastapi), 48 from Rich (Textualize/rich), 13 from Requests (psf/requests), and 1 from HTTPX (encode/httpx), structured with standard SWE-bench fields (instance_id, repo, base_commit, patch, test_patch, problem_statement, hints_text) (;). Furthermore, the main repository provides a dedicated, autonomous SWE-bench benchmark subsystem under src/qb_cubie/benchmarks/swebench/ implementing the swebench-v1 contract and direct import pipelines for official Hugging Face datasets (SWE-bench/SWE-bench, SWE-bench_Lite, and SWE-bench_Verified) (;). Because the three tasks investigated here were selected for test-guided agent development from public repositories, the pilot findings do not represent an official SWE-bench Verified leaderboard entry ().

Data Documentation, 2026Submission Audit, 2026cubieBenchmark Guide, 2026Submission Audit, 2026Single-Agent Report, 2026

A manual domain audit of 39 tasks documented in the candidate catalog (**Candidate Audits, 2026**) classified 26 tasks as qualified candidates, 9 as conditional, 3 as deferred, and 1 as excluded (with 90 tasks remaining semantically unaudited). From the audited pool, three tasks spanning distinct repositories and primary task types were frozen with Seed 42 for the pilot series (**Single-Agent Report, 2026**):

Task ID	Repository & Type	Required Functional Modification	FAIL_TO_PASS	PASS_TO_PASS
fastapi_9425	FastAPI · API/Schema · Compatibility	Treat stringified return annotation "None" in get_typed_return_annotation as a missing response model so HTTP 204 bodiless responses succeed.	1	8
requests_6589	Requests · Parsing/Encoding · Bugfix	Account for UTF-8 encoded byte length rather than character count when computing string length in super_len (Content-Length path).	1	11
rich_3278	Rich · Parsing/Encoding · Bugfix	Strip private ANSI escape sequences in the range ESC 0 through ESC ? so that they do not corrupt console output.	16	7

Prior to each run, repository snapshots are sanitized: git history and private reference files are stripped, symlinks are verified, and shared dependencies are pre-installed (**Single-Agent Report, 2026**). Agents cannot install packages or access an unrestricted shell; they receive only the public problem_statement. Sections 4 and 5 evaluate all seven completed single-agent pilot archives under experiments/testguided-study/runs/ (42 agent runs) plus one prepared-only archive (20261111-180104), while a 60-second technical smoke check (local/live-pilot-validation, capped at 3 calls) is tracked separately (**Single-Agent Report, 2026**). Section 7 then evaluates the five multi-agent follow-up archives (generations v6–v9, 17 executed runs) (**Multi-Agent Synthesis v6–v9, 2026**).

3. Research Methodology and Zero-Leakage Evaluation Harness

For each task and seed pair, both conditions (**cubie** and **pytest**) start from the exact same sanitized snapshot inside a fresh Docker container with an empty dialogue history (**Single-Agent Report, 2026**). Task order is randomized deterministically by seed, and the starting condition alternates across tasks (in the frozen schedule for 20261111-200641, Cubie starts on FastAPI and Rich, while pytest starts on Requests). Runs execute serially without sharing patches, dialogues, or evaluation signals. Both conditions share the exact same model, core instructions, structured JSON tools, context limits, and wall-clock budgets, differing solely in the pre-installed **qb-cubie 0.0.3** wheel (installed into system Python in Dockerfile.cubie and verified before launch via a preflight assertion) and the framework guide (**Study Guides, 2026**).

3.1 Independent Container Evaluation and Host-Environment Provenance

During an episode, both production code and visible tests remain editable by the agent within its permitted role rules (**Single-Agent Report, 2026**). Upon completion or timeout, the full patch and all test edits are archived, and a **fresh, shared evaluator container without Cubie** is launched. All modifications to visible test files and protected runner configurations are stripped from the evaluated patch so that only the pure production diff is applied and executed against the pre-frozen private FAIL_TO_PASS and PASS_TO_PASS regression tests (**Single-Agent Report, 2026**). Consequently, an episode is counted as resolved if and only if the isolated production patch passes all private target and regression tests—even if the agent stalled or timed out without calling submit.

Clarification on Repository Autonomy and Host Wrapper Provenance: The **cubie** project (pyproject.toml) is completely self-contained and has no package dependency on external competition harnesses such as swegemma (**Submission Audit, 2026**). During early single-agent runs (v1–v5) on the developer workstation, the local shell wrapper (./study) resolved a Python interpreter from a neighboring local directory (../devagent-comp/.venv/bin/python) that provided container-bridging utilities (swegemma 0.2.11), and early preparation metadata (DATEN.md, candidate-selection.json) recorded those local workstation paths (/home/symphonie/PycharmProjects/devagent-comp/...) (**Submission Audit, 2026**). Starting with the multi-agent architecture (generations v6–v9 in testguided_study/team_workspace.py and testguided_study/team_v9.py), agent workspace orchestration operates completely autonomously via native git subprocesses and standard Docker containers without invoking swegemma (**Submission Audit, 2026**). All frozen run archives under experiments/testguided-study/runs/ are fully preserved in the repository for independent inspection and re-analysis.

3.2 Frozen Runtime Configuration and Evidence Verification

Across all eight single-agent archives under runs/, **5,563 SHA-256 hash references** covering input manifests, preflight files in ready.json, and run manifests were audited; 100% matched without discrepancy (**Single-Agent Report, 2026**). The primary endpoint is the paired resolution rate difference (Cubie minus pytest), accompanied by an exploratory 95% bootstrap confidence interval resampling tasks with replacement (10,000 draws). The frozen environment for the v5 baseline pilot (20261111-200641) is summarized below:

Parameter	Frozen Specification in Archive 20261111-200641
Model & Serving	gemma4:12b (11.9B, GGUF, Q4_K_M), served locally via Ollama 0.40.2
Model Digest (SHA-256)	4eb23ef187e2c5462566d6a1d3bbbc2f1346d0b4327cbb66d58fffbcc9b2b05c
Sampling & Generation	Seed 42, Temperature 0.2, think=true, num_ctx=32768, num_predict=8192
Time & Call Budgets	900 s agent runtime per attempt; 90 s single command; 120 s private verification; unlimited calls
Context & Output Caps	Configured: 80,000 bytes context / 12,000 chars tool output; effective runner capacity: 40,090 bytes

Parameter	Frozen Specification in Archive 20261111-200641
Agent Core & Guides	testguided-core-v5 paired with pytest-guide-v5 or cubie-guide-v5
Container Environment	Python 3.13.18, pytest 9.2.0; Cubie image with qb-cubie 0.0.3; isolated Cubie-free evaluator container

4. Single-Agent Evolution and Results of the Core Pilot Series (v1–v5)

Following initial container replay validations (**Validation Report, 2026**), the single-agent architecture progressed through five frozen iterations (**Agent Design v1–v5, 2026**): from an early unversioned core (600 s, max 24 calls, 16,384 / 4,096 tokens), to **v2** (explicit working ledger, anchored replacements, stall recovery, and output repair), **v3** (action-coupled notes, evidence-based context, separate path/content search, baseline counter-checks, and EBT runtime tracking; expanded in revised v3 to 900 s, unlimited calls, and 32,768 / 8,192 tokens), **v4** (runner-bound file hashes instead of model-supplied hashes, small-diff guidance, explicit oracle checks, and restricted read recovery), and **v5** (mandatory structured decision points for reproduction analysis, post-failure diagnosis, and pre-submission review, combined with enabling think=true).

4.1 Complete Overview of Single-Agent Pilot Archives (November 11, 2026)

Archive ID	Core & Thinking	Budget (Time / Calls · Ctx / Out)	pytest Resolved	Cubie Resolved
132401	early · think=false	600 s / 24 · 16,384 / 4,096	0 / 3	0 / 3
160158	early · think=false	600 s / 24 · 16,384 / 4,096	0 / 3	0 / 3
163110	v2 · think=false	600 s / 24 · 16,384 / 4,096	**1 / 3**	0 / 3
171515	v3 · think=false	600 s / 24 · 16,384 / 4,096	0 / 3	0 / 3
180104	v3 · think=false	prepared only (no agent runs)	—	—
180702	v3 rev. · think=false	900 s / unlim. · 32,768 / 8,192	0 / 3	**1 / 3**
192412	v4 · think=false	900 s / unlim. · 32,768 / 8,192	**1 / 3**	0 / 3
200641	v5 · think=true	900 s / unlim. · 32,768 / 8,192	0 / 3	0 / 3

Note on pair completeness: Six of the seven completed pilots yielded 3 complete task pairs (0 unavailable). In archive 180702, 2 pairs were complete and 1 pair (requests_6589 in the Cubie arm) was historically marked unavailable due to a patch-induced syntax error during evaluation collection, yielding 41 available evaluations and 1 unavailable evaluation across 42 total runs (**Single-Agent Report, 2026**).

4.2 Verified Resolutions, EBT Non-Utilization (90.5%), and Baseline Pollution

- **v2 Resolution (Archive 163110, requests_6589 with pytest):** The pytest agent resolved requests_6589, demonstrating a focused red–modification–green cycle and submitting cleanly (**Pilot Analyses, 2026**).
- **Revised v3 Resolution & Exit-Code-4 Sensitivity (Archive 180702, fastapi_9425 with qb-cubie):** In run 0001, the Cubie agent modified get_typed_return_annotation to handle "None", authored and executed genuine EBT contracts (6 EBT executions and evaluations, 5 passed across test runs), and submitted a privately verified solution (**Single-Agent Report, 2026**). In the same archive, the Cubie agent on requests_6589 (run 0004) executed 3 EBT contracts but introduced an indentation error into production code, causing the private evaluator to fail during collection with pytest exit code 4. Historically, run 0004 was recorded as resolved=null and available=false, yielding a paired effect of **+50.0 percentage points** (bootstrap CI [0; +100]) across two complete pairs. Counting this agent-induced syntax failure transparently as an unsuccessful resolution yields three complete pairs and a descriptive difference of **+33.3 percentage points** (**Single-Agent Report, 2026**).
- **Extreme EBT Non-Utilization Across v1–v5 (19 of 21 Runs Without EBT):** Archive 180702 (runs 0001 and 0004) represents the **only pilot in the entire v1–v5 series where EBT contracts were actually executed**. Across all 21 Cubie runs in v1–v5, EBT was executed in just 2 runs (9.5%) and remained completely unused in 19 runs (90.5%), including 0/3 runs in v4 and 0/3 runs in v5 (**Single-Agent Report, 2026**; **Submission Audit, 2026**). For example, in archive 163110 (run 0005, Rich), the Cubie agent imported EBT and WorldState but wrote plain pytest assertions in the test body; in archive 200641 (run 0004, Requests), it edited tests/test_utils.py using standard assert statements without importing Cubie.
- **v4 Resolution Without Process Integrity (Archive 192412, fastapi_9425 with pytest):** The production patch passed all 9 private tests (−33.3 percentage points, bootstrap CI [−100; 0]), but the agent overwrote a visible 1,114-line test file with 45 lines of broken tests, ending as stalled without submission (**Pilot Analyses, 2026**).
- **Visible Baseline Pollution:** Preflight checks (preflight/visible) revealed substantial pre-existing failures in the full visible suites before any agent edits: FastAPI exhibited 8 collection errors (missing optional dependencies such as pydantic_settings); Requests had 577 passed, 5 failed, and 15 skipped tests; and Rich had 746 passed, **65 failed**, and 24 skipped tests (due in part to a snapshot inconsistency in Column(highlight=...)) (**Technical Analysis, 2026**).

5. In-Depth Analysis of Pilot v5 (Archive 20261111-200641)

In archive 20261111-200641 (core v5, think=true), all six runs completed private evaluation with 0/3 resolutions for pytest and 0/3 for qb-cubie (paired difference 0 percentage points; degenerate bootstrap interval [0; 0], reflecting zero variance across three 0:0 pairs rather than statistical equivalence) (**Single-Agent Report, 2026**):

Run	Task & Condition	Agent Time	Model / Tool Calls	Termination & Final Patch
0001	FastAPI · Cubie	727.4 s	8 / 6	invalid_output · empty patch
0002	FastAPI · pytest	681.4 s	6 / 4	invalid_output · empty patch
0003	Requests · pytest	829.3 s	17 / 15	invalid_output · test additions only
0004	Requests · Cubie	900.1 s	19 / 17	agent_timeout · test edit only
0005	Rich · Cubie	653.5 s	8 / 6	invalid_output · empty patch
0006	Rich · pytest	684.7 s	9 / 7	invalid_output · empty patch

Across 4,476.4 seconds (74.6 minutes) of pure agent runtime (80.3 minutes total study wall time; median agent time 684.7 s for pytest vs. 727.4 s for Cubie), the two arms executed 67 model calls (32 pytest, 35 Cubie) and 55 tool calls (26 pytest, 29 Cubie), with 1 tool error in pytest, 4 tool errors in Cubie, and 0 EBT executions (**Single-Agent Report, 2026**). Complete token counts for pytest totaled 252,363 tokens (198,821 input / 53,542 output); due to a provider timeout on the final call of run 0004, Cubie recorded a partial sum of 254,210 tokens (combined documented total: 506,573 tokens; 403,338 input and 103,235 output).

Reasoning Token Starvation: Out of 67 model calls, **11 responses terminated at the output cap (done_reason=length at 8,192 tokens) with an empty message.content and exclusively internal thinking output** (**Single-Agent Report, 2026**). These 11 truncated calls consumed 90,112 output tokens—**87.3% of all known output tokens** in v5 and 79.6% of total agent runtime (**Codex Analysis, 2026**). In FastAPI and Rich, both arms failed after reading source files before completing a single reproduction analysis or edit.

Micro-Mechanics on requests_6589: In run 0003 (pytest), the agent authored two accurate UTF-8 reproduction tests—super_len("€") == 3 and super_len("Test€") == 7—which failed against the unpatched code with character counts 1 and 5; however, its subsequent production edit failed with edit_conflict/anchor_missing before the final call hit the 8,192-token thinking cap (**Single-Agent Report, 2026**). In run 0004 (Cubie), the agent selected the 4-byte UTF-8 emoji 🚀, but after syntax and anchor rejections wrote the literal '\ud83d\ude80' (two UTF-16 surrogates, length 2) using a plain assert statement without any EBT contract, failing with 2 == 4 before timing out (**Single-Agent Report, 2026**).

6. Synthesis of Root-Cause Hypotheses (Antigravity & Codex) and Qualitative Context

Following the v1–v5 series, two independent AI systems conducted post-hoc root-cause analyses across all 42 runs: **Antigravity** (Gemini-3.8-Flash, Reasoning: High; **Antigravity Analysis, 2026**) and **Codex** (GPT-6.1-Sol, Reasoning: High; **Codex Analysis, 2026**). Both converge on the diagnosis that the 7.1% resolution rate (3/42) and the 0:0 outcome in v5 represent a structural **floor effect**: in 19 of 21 Cubie runs (90.5%), no EBT contract was ever executed because upstream inference, context, and interface barriers terminated the workflow before test methodology could take effect (**Single-Agent Report, 2026**).

6.1 The Five-Tier Bottleneck Cascade (Antigravity: $H_{Cascade}$ and H1–H4)

- **H1 (Token Starvation / Reasoning Displacement):** In v5's shared output budget (num_predict=8192), unbounded thinking displaced JSON action emission (87.3% output token loss; 11/67 responses or 16.4% of all calls in v5). *Falsifiable prediction:* Capping reasoning tokens (e.g., max 2,048) or raising num_predict to ≥ 16,384 tokens reduces length-truncated invalid_output failures below 10% (**Antigravity Analysis, 2026**).
- **H2 (Prior Dominance / Pre-Training Bias):** Trained overwhelmingly on standard pytest syntax, a 12B model under cognitive load defaults to plain assert statements whenever unrestricted file writing (write_file) is permitted, especially on stateful/fixture tasks (Requests, Rich) as opposed to pure functions (FastAPI in 180702). *Falsifiable prediction:* Restricting test authoring to a dedicated contract-creation tool raises EBT execution from 9.5% (2/21) to over 75% (**Antigravity Analysis, 2026**).
- **H3 (Baseline Pollution / Noise Attractor):** Complete failure on rich_3278 (0/14 across all pilots) is driven by the 65 pre-existing failures in the visible test suite acting as a cognitive distractor. *Falsifiable prediction:* Isolating run_tests to the task-specific test path prior to the first patch reduces stalling on Rich by at least 50% (**Antigravity Analysis, 2026**).
- **H4 (Capability Threshold / Model Boundary):** Simultaneously synthesizing domain logic, exact anchor edits, and valid test oracles exceeds the reliability threshold of 12B Q4_K_M. *Falsifiable prediction:* Evaluating a 30B+ or specialized coding model under the exact same freeze increases resolution on the three tasks from 7.1% to above 30% (**Antigravity Analysis, 2026**).

6.2 Workflow Bottleneck Hypothesis & Factorial Program (Codex) and Project Lead Assessment

Codex formulates a complementary **Workflow & Interface Bottleneck Hypothesis** and specifies a four-stage factorial verification program (**Codex Analysis, 2026**): **Study A** (a 2×2 design crossing v4 vs. v5 × think=false/true to deconfound control prompts from reasoning), **Study B** (handle-bound local edit recovery to eliminate anchor_missing), **Study C** (model scaling under identical budgets), and **Study D** (clean vs. polluted visible baseline). Per the campaign qualification rules (**Single-Agent Report, 2026**), both arms must achieve private resolutions with a verified red–green cycle on at least two independent task groups before launching confirmatory hold-out measurements.

Project Lead's Perspective & Qualitative Observations: Separated from the quantitative metrics, the Project Lead notes that the current sample (three tasks, high floor effect) is insufficient for a definitive statistical verdict on **qb-cubie**, hypothesizing that a stronger model and/or refined orchestration is required (**Single-Agent Report, 2026**). In human-in-the-loop development on internal projects, however, **qb-cubie** has already proven highly valuable: EBT contracts are substantially easier for human developers to read, review, and audit for oracle correctness than procedural pytest code, and capable agent configurations can set up precise EBT tests rapidly. Future iterations will measure these readability and review-time endpoints explicitly (**Single-Agent Report, 2026**).

7. Concluding Follow-Up Study: Multi-Agent Generations v6 to v9

Whereas Sections 1–6 reflect the single-agent baseline documented in the November 11 report (**Single-Agent Report, 2026**), a separate follow-up study on November 12 investigated four serial multi-agent generations (**v6, v7, v8, and v9**; **Multi-Agent Synthesis v6–v9, 2026**). All four generations divided the workflow across three specialized roles sharing the public workspace—**Analysis** (orientation, context curation, and final review), **QA** (test suite maintenance and EBT/pytest case authoring), and **Dev** (test execution and production repair)—under identical model and task settings (gemma4:12b, Seed 42, think=false, temperature=0.15, num_ctx=32768, num_predict=8192, up to 1,800 s wall-clock budget) (**Multi-Agent Designs v6–v9, 2026**).

7.1 Quantitative Overview of Multi-Agent Archives (v6–v9)

Generation & Archive	Exec. / Plan	Model / Tool Calls	Tokens (In / Out)	Reached Roles & Termination Cause
v6 (20261112-002038)	6 / 6	66 / 65	422,512 / 16,749	Analysis (100%) · finished: 6× stagnation stop (0/6 resolved)
v7-1 (20261112-013246)	3 / 6	46 / 45	316,839 / 5,649	Analysis (18), QA (27) · failed: post-freeze ValueError (0/3 resolved)
v7-2 (20261112-015457)	1 / 6	81 / 80	638,637 / 5,705	Analysis (100%) · interrupted: 36× find_paths loop (0/1 resolved)
v8 (20261112-030527)	1 / 6	64 / 63	195,452 / 2,302	Analysis (61), QA (2) · interrupted: 30+ cursor_error loop (0/1 resolved)
v9 (20261112-035522)	6 / 6	1,184 / 1,179	6,980,998 / 44,845	Analysis (100%) · finished: 6× budget_exhausted, 974 dup. (0/6 resolved)
Total (v6–v9)	**17 / 30**	**1,441 / 1,432**	**8,554,438 / 75,250**	**Analysis dominant · 0 / 17 privately resolved (0.0%)**

7.2 Architectural Progression across the Four Multi-Agent Generations (v6–v9)

Across all 1,432 executed tool actions in v6–v9, 90.7% consisted solely of search_content (795 calls, 55.5%) and read_file (504 calls, 35.2%), alongside 72 read_evidence calls (5.0%), 52 find_paths calls (3.6%), 2 ask_analysis calls (0.1%), 3 finish_orientation calls (0.2%), 3 create_case calls (0.2%), 1 collect_tests call (0.1%), and 0 finish calls (0.0%) (**Multi-Agent Synthesis v6–v9, 2026**). Examining the four generations reveals how resolving technical errors exposed a deeper controller-interface handoff dilemma:

1. **Generation v6 (Archive 20261112-002038) — Two-Step Handoff Barrier and Silent Scope Misses:** Disabling reasoning (think=false) restored JSON action stability (65 of 66 calls returned compact valid JSON; one 8,192-token regex escape loop in rich_3278 was recovered via output_repairs). However, all 6 runs ended early via controller stagnation stops (45 s to 531 s) inside analysis/orient without a single call to QA or Dev. On requests_6589, searching in requests/utils.py instead of src/requests/utils.py returned silent empty lists (hits: []), and the model never initiated the two-step publish_context followed by handoff protocol, supporting Counter-Hypothesis V6-H8 ().
2. **v6 Pilot Analyses, 2026**
3. **Generation v7 (Archives 20261112-013246 and 20261112-015457) — Context Overload and Tool-Removal Loops:** v7 added a repository start card, explicit scope_not_found candidate suggestions, atomic finish_orientation, and removed early stagnation stops (**Multi-Agent Designs v6–v9, 2026**). While v7-1 briefly reached QA (defining 3 test cases before aborting on a post-freeze modification check), v7-2 suffered from severe prompt pollution: injecting raw JSON telemetry ledgers (with SHA-256 hashes and float metrics) alongside 45 lines of Cubie tutorials inside the Analysis prompt caused the model to lose track of 55 truncated tool exchanges. When recovery temporarily removed search_content from the schema, the model misused the filename glob tool find_paths to search for code ("pattern": "def get_route_info") 32 times in a row (**Multi-Agent Synthesis v6–v9, 2026**).
4. **Generation v8 (Archive 20261112-030527) — Forced Controller Handoff vs. Context Loss and Cursor Traps:** v8 removed the Cubie tutorial from Analysis, introduced a single-step finish action, and implemented a deterministic forced handoff (repeat_count >= 3 activating QA) (**Multi-Agent Designs v6–v9, 2026**). This forced handoff successfully activated QA, which issued two targeted ask_analysis queries (job-12, job-20). In job-12, Analysis located the exact target functions in fastapi/dependencies/utils.py (lines 253–280). However, because the model did not manually invoke update_context before finishing, the discovered files were lost (materials: [] in job-20). Simultaneously, omitting the explanation of the pagination parameter cursor caused the model to pass "cursor": "0" or file paths, triggering over 30 consecutive cursor_error rejections (**Multi-Agent Synthesis v6–v9, 2026**).
5. **Generation v9 (Archive 20261112-035522) — Infrastructural Perfection vs. Controller-Prompt Interface Lock-In:** v9 eliminated update_context via transactional Zero-Admin Memory, removed cursor from all model schemas, granted all roles symmetric read/search access, rendered prompts as clean four-section Markdown, and removed v8's forced handoff (**Multi-Agent Designs v6–v9, 2026**). Over 1,184 model calls and 1,179 tool calls (6.98M input tokens), infrastructural errors dropped to , and Analysis located the target files within steps 0–13 across all tasks (). However, three coupled harness design factors prevented any transition out of orient: (a) the deterministic forced handoff from v8 was removed in team_v9.py; (b) the finish tool was described in the Analysis prompt primarily with review parameters (verdict=verified/rework and next_role), while supplying a verdict during orient raised ValueError('Orientation cannot verify a repair') (team_v9.py:381), causing the model to invoke finish across 1,184 steps and fall into 974 duplicate actions (82.6%); and (c) because team_v9.py:615 injects the framework guide (if self.role == 'qa': system += '\nFramework guide:\n' + self.guide) exclusively upon entering QA, and QA was never reached, (;).

6. **0 cursor errors, 0 tool errors, and 0 context overflows[v9 Pilot Analysis, 2026]zero timesthe qb-cubie framework instructions were never presented to the model in any v9 run[v9 Pilot Analysis, 2026][Submission Audit, 2026]**

8. Overall Conclusion

Across nine iterative agent generations (59 executed runs: 42 in single-agent pilots v1–v5 and 17 in multi-agent pilots v6–v9), this study delivers an audited, zero-leakage benchmarking methodology for evaluating developer tools like **qb-cubie** on SWE-bench tasks alongside a detailed empirical taxonomy of the bottlenecks governing compact 12B open-weights models (**Single-Agent Report, 2026**; **Multi-Agent Synthesis v6–v9, 2026**). In the single-agent series (v1–v5), flat architectures achieved three privately verified repository repairs—including a proof-of-concept resolution executing six EBT contracts in archive 180702—yet in 90.5% (19/21) of Cubie runs no EBT contract was ever executed due to pre-training pytest priors, edit-anchor friction, and v5's reasoning token starvation (87.3% output token loss). In the multi-agent series (v6–v9), refining the workspace infrastructure eliminated all tool, cursor, and memory errors by v9, but removing forced controller transitions while overloading the orientation finish schema locked the agent in the Analysis role before the QA-scoped **qb-cubie** guide could even be injected (**Submission Audit, 2026**). Lifting agent execution above this operational floor via unambiguous phase-transition contracts, dedicated EBT tool affordances, and higher-capacity models forms the direct empirical roadmap for confirmatory evaluations of **qb-cubie**.

9. References and Frozen Study Artifacts

1. **Single-Agent Report (2026):** Wirzba, D. *qb-cubie in einem testgeführten Programmieragenten: Methodik, Entwicklung und explorative Pilotbefunde* (experiments/testguided-study/PAPER.md, data cutoff: November 11, 2026; covers single-agent generations v1–v5 through archive runs/20261111-200641).
2. **Protocol (2026):** *Studienprotokoll der gepaarten testgeführten Agentenstudie* (experiments/testguided-study/PROTOKOLL.md).
3. **Data Documentation (2026):** *Herkunft und Struktur des SWE-bench-Aufgabenpools* (experiments/testguided-study/DATEN.md).
4. **Candidate Audits (2026):** *Fachliche Einzelprüfungen und Kandidatenkatalog* (experiments/testguided-study/KANDIDATEN.md and experiments/testguided-study/candidate-catalog.json).
5. **Study Guides (2026):** *Eingefrorene Agentenanleitungen für qb-cubie und pytest* (experiments/testguided-study/guides/cubie.md and experiments/testguided-study/guides/pytest.md).
6. **Validation Report (2026):** *Technische Durchstich- und Replay-Validierung der Studienbrücke* (experiments/testguided-study/VALIDIERUNG.md).
7. **Agent Design v1–v5 (2026):** *Entwicklungshistorie der Agentenkerne v1 bis v5* (experiments/testguided-study/AGENTENDESIGN.md, AGENTENDESIGN-V4.md, and AGENTENDESIGN-V5.md).
8. **Pilot Analyses (2026):** *Detaillierte Auswertungen der Pilotläufe v2 bis v4* (experiments/testguided-study/PILOTANALYSE-20261111.md and PILOTANALYSE-20261111-192412.md).
9. **Technical Analysis (2026):** *Analyse der sichtbaren Ausgangstests und Snapshot-Vorbelastungen* (experiments/testguided-study/TECHNIKANALYSE-20261111-171515.md).
10. **Antigravity Analysis (2026):** *Post-hoc-Ursachenanalyse und Kaskaden-Hypothesensystem (Gemini-3.8-Flash, Reasoning: High)* (experiments/testguided-study/HYPOTHESEN-URSACHENANALYSE-20261111.md).
11. **Codex Analysis (2026):** *Post-hoc-Ursachenanalyse und Workflow-Prüfprogramm (GPT-6.1-Sol, Reasoning: High)* (experiments/testguided-study/URSACHENANALYSE-CODEX-20261111.md).

12. **Benchmark Guide (2026):** *Benchmarks: Einstieg, eigene Agenten und Messbetrieb inkl. SWE-bench-Integration* (docs/GUIDE_Benchmarks.md and src/qb_cubie/benchmarks/swebench/, November 11, 2026).
13. **Contract Semantics (2026):** *Verbindliche Vertragssemantik von Cubie 0.0.3* (docs/GUIDE_Contract-Semantics.md, aligned with src/qb_cubie/ebt.py, protocols.py, and rbt.py).
14. **Quality Assurance Guide (2026):** *Qualitätssicherung mit Cubie: Beobachtung, Abdeckung und Beweis* (docs/GUIDE_Quality-Assurance.md).
15. **EBT Guide (2026):** *Example-Based Testing: Ausgangsdaten, Zielmuster und Unterspezifikation* (docs/GUIDE_Example-Based-Testing.md).
16. **EBT Usage Pattern (2026):** *EBT-Nutzungsmuster und ausführbare Vertragsbeispiele* (docs/GUIDE_EBT-Usage-Pattern.md).
17. **Arena Management (2026):** *Arena-Management: TempFS, PyVenv, Hydrierungsgrenzen und Laufzeitlimits* (docs/GUIDE_Arena-Management.md).
18. **RBT Usage Pattern (2026):** *RBT-Nutzungsmuster: Interaktive und automatisierte Reviewer* (docs/GUIDE_RBT-Usage-Pattern.md).
19. **Linter Guide (2026):** *Linter: AstLinter, Pylint/MyPy-Verträge und CstLinter-Regeln* (docs/GUIDE_Linter.md).
20. **Technical Audit (2026):** *Kritisches Gutachten zu qb-cubie* (./GUTACHTEN.md in the project root directory of cubie, November 11, 2026, commit 5e50d22f80db90c4e8ae6e3aeedab6acdb981230, version 0.0.3).
21. **Multi-Agent Designs v6–v9 (2026):** *Konzepte und Implementierungen der Generationen v6 bis v9* (experiments/testguided-study/AGENTENDESIGN-V6.md, AGENTENDESIGN-V7.md, IMPLEMENTIERUNG-V7.md, AGENTENDESIGN-V8.md, AGENTENDESIGN-V9.md, and IMPLEMENTIERUNG-V9.md).
22. **v6 Pilot Analyses (2026):** *Deskriptive Analysen des v6-Piloten 20261112-002038* (experiments/testguided-study/PILOTANALYSE-V6-20261112-002038-CODEX.md and PILOTANALYSE-20261112-002038.md).
23. **v9 Pilot Analysis (2026):** *Deskriptive Analyse der Agenten-Generation v9, Laufarchiv 20261112-035522* (experiments/testguided-study/PILOTANALYSE-V9-20261112-035522.md, Antigravity / Gemini-3.8-Flash).
24. **Multi-Agent Synthesis v6–v9 (2026):** *Deskriptive Analyse und vergleichende Gesamtschau der Agenten-Generationen v6, v7, v8 und v9* (experiments/testguided-study/ANALYSE-GENERATIONEN-V6-V9.md, Antigravity / Gemini-3.8-Flash, November 12, 2026).
25. **Submission Audit (2026):** *Kritische Analyse und Mängelrüge zur Paper-Submission „qb-cubie and Floor-Effect Analysis“* (experiments/testguided-study/KRITISCHE-ANALYSE-PAPER-SUBMISSION.md, Antigravity / Gemini-3.8-Flash, November 12, 2026).